Google Summer of Code 2026 Proposal
AI-Assisted Log Diagnosis & Root-Cause Detection

ArduPilot

Applicant: LuciferK47
Mentor: Nathaniel Mailhot
Project Size: 350 hours (Large)
Difficulty: Hard

---

1 Abstract

I propose an open-source, learning-based diagnostic system that automatically analyzes ArduPilot DataFlash logs to pinpoint the root causes of flight failures. Currently, diagnosing a crash requires deep human expertise to manually correlate diverging telemetry graphs. While automating this is highly desirable, existing tools relying on hardcoded parameter checks inevitably break when firmware updates rename or drop log fields over time. One challenge is the instability of telemetry schemas across firmware versions and the lack of labeled datasets, which has historically made learning-based approaches difficult to deploy reliably.

To resolve this, I propose a data pipeline built around a Version-Agnostic Feature Extraction Layer. By dynamically abstracting telemetry (e.g., resolving a generic "roll error" concept to `ATT.ErrRP` if present, or computing it from `ATT.DesRoll - ATT.Roll` if absent), the system will support cross-version logs without requiring the user to specify their firmware version. I have verified the feasibility of parsing DataFlash logs using `pymavlink` and explored SITL-based fault injection workflows to ground this design.

This robust feature set will feed a hybrid Rule Engine + ML Classifier. The GSoC timeline focuses on scaling this architecture to reliably detect 8 specific failure classes (including EKF divergence, motor loss, and GPS glitches) using a combination of SITL-generated faults and real-world validation data. The planned deliverable includes a Python CLI, a FastAPI REST service, and a curated labeled dataset.

2 Limitations of Existing Tools (`LogAnalyzer`)

ArduPilot's legacy `LogAnalyzer` executes static threshold checks (e.g., flagging "Compass variance high"). Upon reviewing its source logic, I identified structural limitations that this proposed project aims to resolve:

1. **Symptom Detection vs. Root Cause Diagnosis:** `LogAnalyzer` flags symptoms (e.g., *VCC brownout*), but generally lacks the stateful logic to correlate concurrent streams to identify the underlying mechanical fault.
2. **Context Blindness:** Rule thresholds are static. A compass variance of 35% may be benign on a large hexacopter with physically separated sensors, but critical on a compact 250mm quad. A machine learning approach naturally conditions its predictions on the vehicle's unique parameters.
3. **Version Fragility:** `LogAnalyzer` relies heavily on hardcoded field parsers, which can throw errors or silently ignore data on newer 4.x logs where message structures have evolved.

3 System Architecture

The proposed diagnostic system processes a `.bin` file through a pipeline consisting of six sequential stages. In practice, raw logs are first ingested and resampled to a unified frequency. Next, a feature abstraction layer maps raw telemetry names into version-agnostic features, which are then evaluated by a deterministic rule engine. If the rules do not capture the fault, the features are processed by a machine learning classifier, and historical precedents are fetched via a semantic retrieval layer before merging everything into a final JSON report.

3.1 1. Log Ingestion

The system uses `pymavlink.DFReader` to iteratively read the log. It extracts numerical streams (`ATT`, `RCOU`, `VIBE`, `NKF4`) and the vehicle `PARM` table. Because sensors log at varying asynchronous frequencies, all continuous data is resampled via linear interpolation into a unified 10 Hz `pandas.DataFrame`.

3.2 2. Feature Abstraction Layer (Core Innovation)

One challenge when building tools across multiple ArduPilot versions is configuration drift. To address this, I propose a declarative YAML Feature Registry. Instead of hardcoding `ATT.ErrRP`, the logic defines fallback priorities:

```
abstract_metric:  roll_tracking_error
priority_1:  ATT.ErrRP
fallback_compute:  abs(ATT.DesRoll - ATT.Roll)
```

In practice, if firmware 4.5 changes a field name, the core Python parsing code remains untouched; only the configuration file requires a targeted update. This is important because it prevents the tool from breaking easily as the ecosystem evolves. If a fallback cannot be computed entirely, the feature is cleanly marked as `np.nan`. The unified dataframe is then processed using a rolling window (`df.rolling('30s')`), outputting a numerical vector aggregating the `mean`, `std`, `max`, and `min` of the signals over the window.

3.3 3. Failure Pattern Layer (Rule Engine)

The feature vector first passes through a set of explicit, deterministic heuristics. By targeting strict physical thresholds (e.g., severe attitude divergence combined with motor saturation), the highest-confidence physical faults sidestep the ML layer entirely, providing highly auditable verdicts.

3.4 4. ML Classification Layer

Rolling windows not decisively flagged by the rules are passed to an XGBoost classifier. Tree ensembles are well-suited for this setting and are expected to perform robustly given the tabular nature of the data. Furthermore, XGBoost natively routes `np.nan` inputs (vital for the missing fields dropped by the Feature Layer) and generates SHAP-value feature importance, enabling explainable predictions.

3.5 5. Retrieval Layer & 6. Output Layer (Conflict Resolution)

A FAISS-based semantic search fetches the $k = 5$ most similar historical logs based on the numerical feature embeddings.

Conflict Resolution Logic: If the deterministic Rule Engine triggers with $\geq$90% certainty, it short-circuits the ML output. If the rule confidence is low or ambiguous, the ML classifier's

Platt-scaled probability distribution is used. The FAISS retrieval provides precedents to append to the final JSON report.

4 Failure Taxonomy & Target Output

The system focuses on the following primary taxonomy. I plan to refine these during Community Bonding and build specific SITL payload injection scripts to generate training data:

- **Motor/ESC/Propeller Loss:** Correlating `ATT.Roll` vs `ATT.DesRoll` divergence with simultaneous `RCOU.C1..C4` saturation.
- **GPS Glitch / Spoofing:** Analyzing `GPS.HDop` spikes against `NKF4.SP` position innovation variance.
- **Compass Interference:** Measuring correlation between `MagX/Y/Z` drift and throttle payload (`CTUN.ThO`).
- **EKF Divergence:** Lane switching (`NKF4.SV/SP/SM` ratios > 1.0) lacking specific single-sensor faults.
- **Excessive Vibration:** Clipping events on `VIBE.Clip0` coupled with velocity variance.
- **Power/Battery Sag:** `BATT.Volt` dropping beneath `BATT_LOW_VOLT` strictly under heavy load.

4.1 Example Target Output Payload

The proposed pipeline design aims to output a structured JSON response similar to the following when fed an `ENGINE_FAIL` log:

```
{
  "status": "CRITICAL_FAILURE",
  "root_cause": "motor_loss",
  "confidence": 0.94,
  "evidence": [
    {
      "signal": "ATT.Roll vs ATT.DesRoll",
      "metric": "roll_tracking_error_max",
      "peak_value": 28.4,
      "time_us_window": [140232100, 145000000]
    }
  ],
  "retrieval_precedent": ["sim_quad_fault_041.bin (Distance: 0.12)"],
  "suggested_fix": "Inspect Motor 3 and its ESC for physical damage."
}
```

5 Deliverables & MVP Definition

1. **Hybrid Diagnostic Engine:** Python package (`ardupilot-logdiag`) containing the Feature Registry, Rule Engine, and ML pipeline.
2. **Interfaces:** A CLI (`logdiag analyze`) and FastAPI backend (`POST /analyze`).
3. **Labeled Dataset:** SITL fault-injection scripts generating labeled segments across the target failure classes.
4. **Documentation & Evaluation:** User docs and an evaluation report demonstrating classification robustness.

MVP Definition: By midterm, the system will reliably ingest `.bin` files via the CLI, extract version-agnostic features into a `pandas.DataFrame`, and detect at least 2 high-confidence failure classes (e.g., motor loss, vibration) with consistent rule-based detection on SITL-generated logs.

6 Current Progress

To de-risk the proposal and validate feasibility, I have completed the following preliminary work:

- Successfully parsed ArduPilot DataFlash logs using `pymavlink` and extracted key telemetry streams (`ATT`, `RCOU`, `NKF4`).
- Implemented initial data resampling experiments in `pandas` to unify asynchronous log streams.
- Explored SITL-based fault injection workflows (e.g., `SIM_ENGINE_FAIL`) to understand how failures manifest in logs.
- Studied the `LogAnalyzer` codebase to identify structural limitations and inform the proposed architecture.

7 Implementation Timeline (12 Weeks)

Total runtime is estimated at ∼350 hours. The timeline prioritizes the pipeline infrastructure early to mitigate integration risks.

Period	Milestones & Goals
Community Bonding (Weeks -2 to 0)	
	Finalize diagnostic taxonomy with mentor. Prototype the `pymavlink` parser abstractions. Finalize the abstract target fields for the YAML Feature Registry. Build familiarity with SITL parameter injection.
Phase 1: Pipeline & Data (Weeks 1–3) ∼87h	
Week 1–2	Develop the YAML Feature Registry and runtime fallback resolver. Validate that extraction works on real logs spanning firmware 4.1 to 4.5.
Week 3	Write SITL fault-injection scripts (e.g., `SIM_ENGINE_FAIL`) to generate a foundational dataset of $\geq$ 400 segments.
Phase 2: Baseline Models & Midterm (Weeks 4–6) ∼87h	
Week 4–5	Code the Rule Engine framework. Train baseline XGBoost classifiers utilizing `np.nan` imputation for missing features.
Week 6	Implement FAISS retrieval index. Finalize target JSON formatting.
Result	**Midterm Delivery: A fully functional CLI MVP capable of detecting 2+ failure classes via the Rule Engine over 10Hz resampled logs.**
Phase 3: Refinement & Explanation (Weeks 7–9) ∼87h	
Week 7–8	Integrate SHAP tree-explainers to accurately map feature importance back to the specific raw `TimeUS` bounds in the log.
Week 9	Expand dataset ($\geq$ 500 total). Evaluate models via an 80/20 stratified train/test split.
Phase 4: Interfaces & Finalization (Weeks 10–12) ∼87h	
Week 10–11	Wrap system within FastAPI REST endpoint. Conduct missing-field robustness evaluations to simulate firmware mismatches.
Week 12	Clean repository. Handover user, developer, and API documentation.
Result	**Final Evaluation Delivery ready for WebTool integration trials.**

8 Evaluation & Mitigation of Simulator Limitations

Evaluation Methodology: System performance will be evaluated on a stratified 80/20 train/test split. Target macro-F1 ≥ 0.7 on held-out data. Additionally, probability confidence calibration will be quantified using the Expected Calibration Error (ECE).

Risks and Mitigation:

- **Data Scarcity & Simulator Reality Gap:** The primary ML training set will be synthetically generated via SITL injections (e.g., `SIM_MAG_MOT_*`). However, while SITL generates clean faults, real-world logs contain coupled noise. To prevent overconfidence in perfectly isolated simulator data, the model will be actively validated against a curated subset of real-world ArduPilot Discourse logs.
- **Firmware Drift:** The most common failure mode for analytic tools. **Mitigation:** Handled natively by the architectural separation of the abstract Feature Registry (YAML) and raw telemetry parsing.
- **Fallback Plan:** If ML performance is insufficient, the system will still deliver value via the deterministic Rule Engine, ensuring the project produces a usable diagnostic tool regardless of model accuracy.

9 Future Work

Post-GSoC expansions include: (1) Integrating the REST API natively into the ArduPilot WebTools suite, (2) Expanding the Rule Engine/Taxonomy to cover Plane-specific aerodynamics (e.g., TECS divergence) and Rover faults, and (3) Implementing a continuous-learning feedback loop within WebTools.

10 About Me

I am a final-year undergraduate studying Electronics and Communication Engineering. My background bridges hardware systems with active Python ML engineering, making me well-prepared to execute this plan.

- **Systems Intuition:** My ECE degree provides the physical signal-processing intuition required to construct interpretable telemetry features (e.g., understanding how vibration couples into EKF navigation errors) before passing data to ML models.
- **Practical ML Engineering:** I have built end-to-end data pipelines using `pandas`, Random Forests, XGBoost, and FAISS vector databases. I understand the stringent limitations of machine learning datasets and actively prioritize transparency (e.g., using SHAP) over black-box complexity.
- **Preparation:** I have spent time over the past few weeks studying the ArduPilot log diagnostic documentation, bootstrapping SITL, and analyzing the `LogAnalyzer` source logic to ensure the architecture I propose addresses a persistent problem in the ecosystem.

I am driven to ship reliable and testable code. While I expect to rely on my mentor for guidance regarding deep flight-dynamics nuances, my engineering capabilities are robustly suited to bring this pipeline to fruition.